An Authoritative STM32 Tic-Tac-Toe Server with a Checksummed Serial Protocol

Hazel Kunyavapa (KNYCHI008) Oswika Mukoma (MKMOSW001)
EEE3095S CS Project 2026, Department of Electrical Engineering
University of Cape Town

***Abstract*—We built a Tic-Tac-Toe game in which an STM32F051 is the only referee and a Python Tkinter program is an untrusted display. The board holds the whole game in 10 bytes and changes it only after every check of a request has passed, so a rejected request cannot alter the game. Messages are ASCII lines of at most 32 bytes with an XOR checksum; the server answers each frame with exactly one reply (when the client waits for it), rejects oversize, corrupt, unknown and stalled frames with specific `NACK` codes, and recovers on the next valid frame. We test the same suite of 100 pytest cases against a PC mock, against the real C logic compiled for the PC, and against the board: all 100 pass on the board in 91.96 s. Fifteen deliberately injected faults are all caught by the suite, and an exhaustive PC run of the rules checks 50.5 million cases, including 6.86 million illegal moves that left the game unchanged.**

I. Introduction

In a networked game, whoever holds the rules can cheat if the other side is trusted. The brief [1] therefore puts the referee on an STM32F051 microcontroller and treats the PC program as an untrusted display. Our design guarantees one property: *a rejected request never changes the game*. The STM32 owns the board, the turn and the win test, and the PC only sends requests and draws what the board confirms.

We chose Tic-Tac-Toe because its 8 winning lines and 9 cells can be checked exhaustively, which lets us spend the effort on the protocol and the tests, as the mark scheme rewards. The PC client uses Python's Tkinter [2]: it ships with Python, so a marker can run it without installing a toolkit, and its `after()` polling suits our worker-thread design (Sec. III).

Three decisions shape the system. (i) The rules are portable C with no hardware calls, so the same source runs on the Cortex-M0 and on a PC. (ii) Messages are short ASCII lines with an XOR checksum, and every frame gets exactly one reply (for clients that wait for replies). (iii) One pytest suite of 100 tests runs unchanged against a Python mock, the real C logic on the PC, and the board. We reused the course base [3] for start-up code and HAL drivers, and we used an AI assistant [4] to draft code, tests, diagrams and text; both authors reviewed and ran every module.

Task allocation. [Hazel Kunyavapa and Oswika Mukoma: write who did what (for example firmware and protocol; GUI and test harness) and what you did together.]

Structure. Section II defines the protocol, Section III gives the firmware and GUI design, Section IV explains three code excerpts, Section V argues that the server stays authoritative, Section VI covers bad input, Section VII reports tests, and Section VIII concludes.

TABLE I
Framing rules and the server's response to each violation.

Rule	Server behaviour
Terminator	LF ends a frame; one CR before it is dropped
Maximum length	32 bytes before the LF; the 33rd byte gives `NACK:TOOLONG` at once, the rest is skipped silently to the next LF
Incomplete frame	500 ms without a byte: `NACK:TIMEOUT` (silent while skipping)
Invalid character	Empty frame or byte outside 0x20–0x7E: `NACK:BADCMD`
Checksum	Malformed or wrong `*HH`: `NACK:BADSUM`

II. Requirements and Message Protocol

A. Lines of at most 32 bytes, each answered exactly once

The link is USART1 on PA9 (TX) and PA10 (RX) through the ST-Link virtual COM port, 115200 baud, 8N1, no flow control. A *frame* is `body[*HH]` followed by LF (Table I). The body is printable ASCII; `HH` is the XOR of all body bytes as two uppercase hex digits. Replies always carry it; requests may omit it so a person can type into a terminal, but a wrong one is rejected. Every frame, whether complete, oversize or timed out, gets exactly one reply, and the server never speaks unprompted.

A frame is checked in a fixed order and the first failure wins: length, printable, checksum, then command syntax. Only a frame that passes all four reaches the game state machine, so no framing error can touch game state.

B. The board is nine characters

The board is nine characters, cells 0–8 left to right, top to bottom: `.` empty, `X`, `O`. `XO.XO....` has X at cells 0 and 3 and O at 1 and 4. X always moves first, and the side to move is X when the counts are equal, otherwise O, so a client can derive it from the board alone.

C. Six commands, each defined in every state

Table II lists the commands. The state table is total: all 18 command/state pairs have a defined reply, and a valid command in the wrong state gets `NACK:NOGAME` when it needs a running game and `NACK:STATE` otherwise. A game started or reset with the board's button sends nothing; the client

TABLE II
COMMAND SET. † MARKS A DEPARTURE FROM THE BRIEF.

Request	Accepted in	Success response
CONNECT	IDLE	OK
NEW	IDLE, GAME_OVER	OK:BOARD:.........
MOVE:<i>[:<m>]†	IN_PROGRESS	OK:BOARD:<9>, or WIN:<m>:BOARD:<9>†, or DRAW:BOARD:<9>†
STATE	IN_PROGRESS, GAME_OVER	BOARD:<9>
RESIGN	IN_PROGRESS	OK:GAMEOVER
DISCONNECT	IDLE†, GAME_OVER	OK

TABLE III
EVERY ERROR RESPONSE AND WHEN THE SERVER SENDS IT.

Response	Sent when
NACK:TOOLONG†	Frame longer than 32 bytes
NACK:TIMEOUT†	Partial frame, 500 ms without a byte
NACK:BADSUM†	*HH malformed or wrong
NACK:BADCMD	Empty, non-printable, unknown command or bad argument (e.g. MOVE:4X)
NACK:NOGAME	Command needs a running game and none exists
NACK:STATE†	Valid command, wrong state (e.g. NEW mid-game)
NACK:RANGE	Index outside 0–8 (9, −1)
NACK:NOTURN	Claimed mark is not the side to move
NACK:OCCUPIED	Target cell not empty

finds out on its next request (MOVE gives NACK:NOGAME) and returns to its start screen.

D. Nine NACK codes, checked in a fixed order

Table III lists every error. Every NACK leaves board, turn and state unchanged. A MOVE that fails several checks reports the first in this order: state, range, turn, occupancy. For example MOVE:9:O on X's turn is NACK:RANGE, so each request has one correct reply.

E. Six departures from the brief, each justified

1) *Terminal replies carry the board* (WIN:X:BOARD:...). The client redraws from one reply instead of paying a STATE round trip.
2) *Optional :<m> on MOVE.* With one client an out-of-turn move cannot otherwise be sent, so NACK:NOTURN would be untestable. The GUI always sends the mark, so a stale client is detected.
3) *Extra codes* TOOLONG, TIMEOUT, BADSUM, STATE, so framing and integrity failures are distinguishable from bad commands.
4) *Checksum optional on requests*, mandatory on replies, to keep terminal testing possible.
5) *DISCONNECT accepted in IDLE*, so a client that connected but never started can leave cleanly.
6) *STATE in IDLE is NACK:NOGAME*, and *NEW in progress is NACK:STATE* (resign first); the button is a local reset with no message.

TABLE IV
MODULES AND WHO OWNS EACH DECISION.

Module	HAL?	Responsibility
game.c	no	10-byte state, validate-then-commit, 8-line win test
server.c	no	byte-driven framer, parser, checksum, dispatch to game.c
uart.c	yes	USART1 RX interrupt, 128-byte ring, polled TX
main.c	yes	loop: drain ring, server_rx, server_tick, button, LED
client.py	no	protocol client, checksum check, GameView (no Tk)
app.py	Tk	canvas, worker thread, score, sound

Figure 1 shows both halves of the protocol. In (a) a game runs from CONNECT to an X win; the move after the win is refused with NACK:NOGAME. In (b) the server refuses an occupied cell, an out-of-range index and a corrupted request, and a following STATE shows the board unchanged.

III. SPECIFICATION AND DESIGN

A. Pure logic is separated from hardware, and the board decides everything

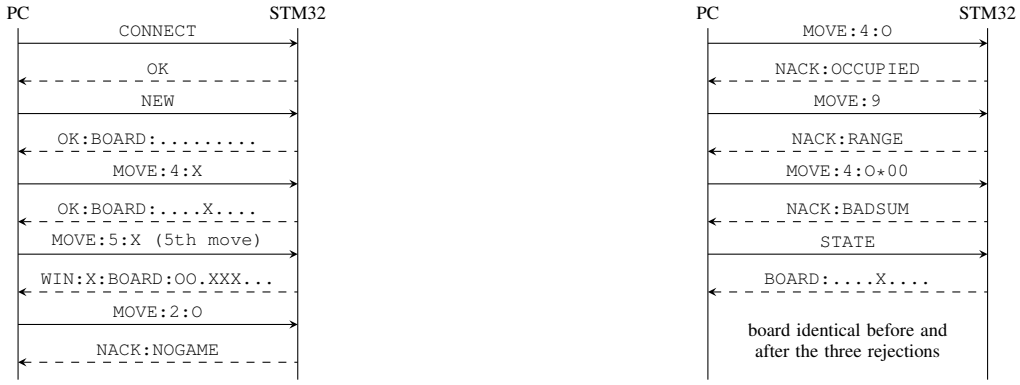
Table IV divides the system. game.c and server.c contain no HAL or CMSIS calls, so the identical files are compiled for the Cortex-M0 and for the PC test builds; only uart.c, main.c and the button/LED code touch registers. The server owns the board, the turn, legality, win detection and the reply text. The client owns only presentation (drawing, score, sound) and the timing of its own requests. It never computes a legal move or a winner, so a buggy or hostile client can at worst be refused.

B. The server FSM validates before it commits

Figure 2 shows the three states. IDLE accepts CONNECT, NEW and DISCONNECT. IN_PROGRESS accepts MOVE, STATE and RESIGN, and GAME_OVER accepts NEW, STATE and DISCONNECT. The MOVE branch runs the checks range, turn, occupancy in that order, each with its own NACK exit that returns to IN_PROGRESS without writing. Only a move passing all three is committed; the win or draw test then decides between IN_PROGRESS and GAME_OVER. The button resets to IDLE from any state. The figure omits the framer errors, which never reach the FSM (Table I).

C. The GUI never draws before the server confirms

Figure 3 shows the client flow. A request is placed on a worker thread, so the Tk main loop never waits on the serial port, and the main loop polls a result queue every 30 ms. On a reply the client first verifies the checksum; a missing or wrong one counts as not received. Only a verified OK:BOARD, WIN, DRAW or BOARD reply redraws the board, and a NACK shows its message and leaves the display as it was. After a win, draw or resign, cells are disabled until NEW. On a lost reply (1 s timeout) the client sends STATE once and redraws from it, because STATE is safe to repeat; if that also fails it reports "server not responding". If a MOVE is answered



(a) X wins (b) rejections (board `....X....`)

Fig. 1. Sequence diagrams (checksums omitted; solid = request, dashed = reply). (a) A scripted X win. (b) Three rejections leave the board unchanged.

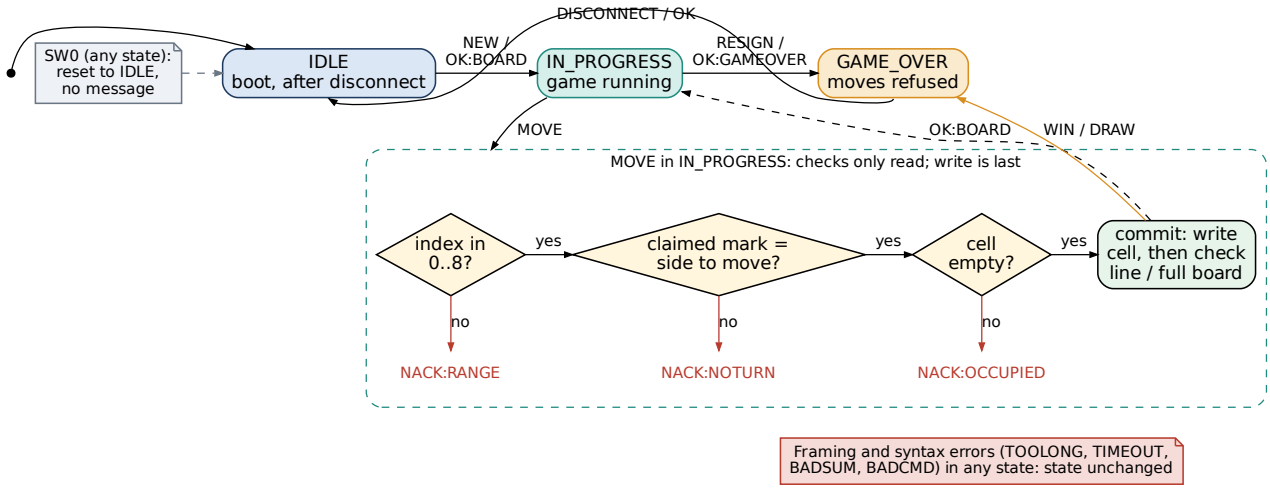


Fig. 2. Server FSM with the `MOVE` validation branch. Transitions read "command [guard] / reply".

`NACK:NOGAME` mid-game, the board was reset by its button, so the client returns to the start screen.

D. The USB cable is the UART link; SW0 and the LED are local

Figure 4 shows the connection. No adapter is used: the on-board ST-Link bridges USART1 to a USB virtual COM port, with TX on PA9 and RX on PA10 (alternate function AF1, datasheet Table 14 [5]). SW0 on PA0 is active low with the internal pull-up; a press resets the game to IDLE. The red LED is on PB10 and the green on PB11, each through 150 Ω (schematic [6]). The LED is off in IDLE, green while X is to move, red while O is to move, and both blink together at 2 Hz (250 ms half-period in code) in GAME_OVER.

IV. Implementation

A. The interrupt only moves bytes; the main loop does the work

At 115200 baud a byte arrives every 87 µs, and the USART has a one-byte receive register, so a second byte overwrites the first if the program is busy elsewhere (overrun). Listing 1 is the whole receive interrupt: it reads `RDR`, stores the byte in a 128-byte ring buffer and returns. The ring has a power-of-two size so the wrap is a mask, and it is written only by the interrupt (`head`) and read only by the main loop (`tail`), so no lock is needed. If the ring is full the byte is counted and dropped rather than waiting. The error flags (overrun, framing, noise) are cleared by writing `ICR`, as described in the reference manual [7], so that one error does not persist. We trimmed the declarations.

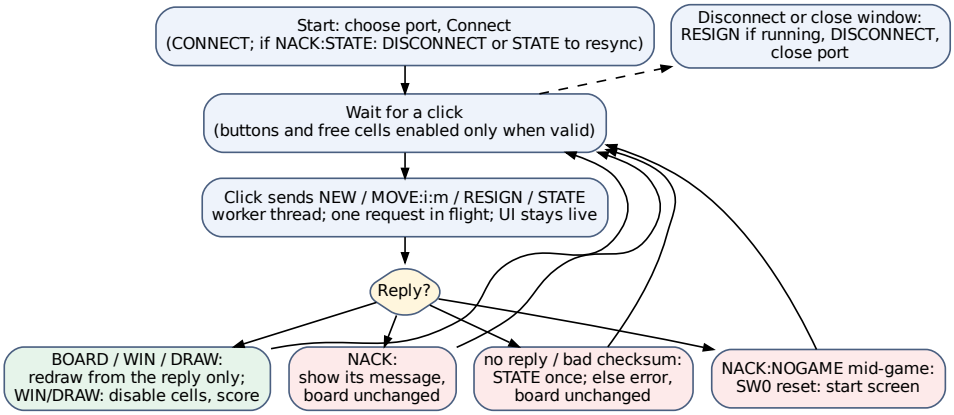


Fig. 3. GUI flowchart: requests run on a worker thread and the display changes only on a verified reply.

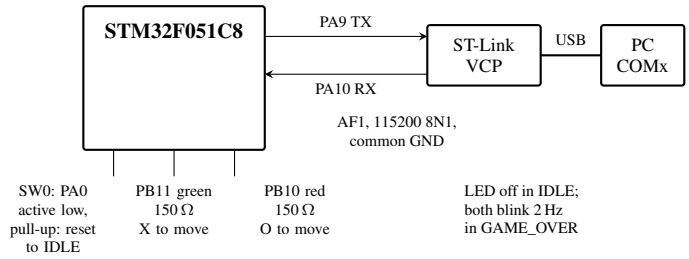


Fig. 4. UART path to the PC and the button and LED pins.

```
void USART1_IRQHandler(void){
  uint32_t isr = USART1->ISR;
  if (isr & (USART_ISR_ORE|USART_ISR_FE|USART_ISR_NE)) {
    if (isr & USART_ISR_ORE) overruns++;
    USART1->ICR = USART_ICR_ORECF | USART_ICR_FECF
                | USART_ICR_NCF;    /* clear flags */
  }
  if (isr & USART_ISR_RXNE) {
    uint8_t b = (uint8_t)USART1->RDR;  /* clears RXNE */
    uint16_t next = (head + 1u) & (RING_SZ - 1u);
    if (next != tail) { ring[head] = b; head = next; }
    else dropped++;           /* full: drop, never wait */
  }
}
```

Listing 1. Receive interrupt and ring buffer (`uart.c`, trimmed).

B. The framer replies at the 33rd byte and resynchronises at the next LF

Listing 2 shows the byte-driven framer. The obvious alternative is to collect bytes until the LF and then test the length, but a sender that never sends LF would then fill the buffer or stall the server. Here the 33rd byte produces `NACK:TOOLONG` immediately and moves to a DISCARDING state that drops bytes silently until the next LF, so an oversize frame gets one reply, not two, and the buffer cannot overflow. Each call writes at most one reply and never waits. The inactivity timer

```
size_t server_rx(server_t *s, uint8_t byte,
   uint32_t now_ms, char *out, size_t cap){
  size_t n = 0;  s->last_ms = now_ms;
  switch (s->rx) {
  case RX_IDLE: case RX_RECEIVING:
    if (byte == LF) {          /* len 0: empty frame */
      n = handle_frame(s, s->buf, s->len, out, cap);
      server_reset_framer(s);
    } else if (s->len >= SERVER_MAX_FRAME) { /*33rd*/
      n = make_reply("NACK:TOOLONG", out, cap);
      s->len = 0;  s->rx = RX_DISCARDING;
    } else { s->buf[s->len++] = (char)byte;
             s->rx = RX_RECEIVING; }
    break;
  case RX_DISCARDING:
    if (byte == LF) server_reset_framer(s);
    break;
  }
  return n;
}
```

Listing 2. Framer (`server.c`, trimmed). At most one reply per call.

is checked in `server_tick`, not with a delay, using the unsigned difference `now - last_ms`. This caused a bug: the main loop first read the time and then drained the ring, so `last_ms` could be newer than `now` and the unsigned subtraction wrapped to about 4×10^9, producing a false `NACK:TIMEOUT`. We found it by reading the code before the first flash and fixed it by reading the clock *after* the drain, as the comment in `main.c` records.

C. The GUI does serial I/O on a worker thread and draws on a timer

A `MOVE` can take up to the 1 s reply timeout, and a Tk window that waits for it stops redrawing and looks frozen. Listing 3 runs each exchange on a daemon thread and passes the result back through a queue; only the Tk thread touches widgets. `poll` drains the queue every 30 ms using `root.after`, which is Tk's thread-safe way to schedule work on its own loop. The three exception paths separate

```
def run_in_thread(self, work):
    self.busy = True; self.refresh()
    def target():
        try:    self.results.put(("ok", work()))
        except ReplyError as exc:
            self.results.put(("err", str(exc)))
        except Exception as exc:  # port gone
            self.results.put(("fatal", str(exc)))
    threading.Thread(target=target, daemon=True).start()

def poll(self):
    try:
        while True:
            kind, payload = self.results.get_nowait()
            self.finish(kind, payload)
    except queue.Empty: pass
    self.root.after(30, self.poll)
```

Listing 3. GUI worker thread and queue polling (`app.py`, trimmed).

```
if (g->state != GAME_IN_PROGRESS) return GAME_ERR_NOGAME;
if (index < 0 || index >= GAME_CELLS)
                                  return GAME_ERR_RANGE;
const char side = game_side_to_move(g);
if (mark != 0 && mark != side)    return GAME_ERR_NOTURN;
if (g->board[index] != '.')       return GAME_ERR_OCCUPIED;
/* ---- commit: first write to *g ---- */
g->board[index] = side;
const game_result_t r = game_evaluate(g->board);
if (r != GAME_RESULT_NONE) g->state = GAME_OVER;
```

Listing 4. Validate-then-commit (`game.c`, comments trimmed).

a protocol error (`ReplyError`, shown as a message), and anything else such as a missing or unplugged port ("fatal", disconnect), so an unexpected failure can never leave the `busy` flag set and lock the board. The `exchange` method sends `STATE` once after a lost reply, as described in Sec. III.

V. Game Logic and Server Authority

A. All state is 10 bytes owned by the STM32

The whole game is one `game_t`: nine cells and a state byte, 10 bytes with no padding. The client never holds it; it receives only checksummed text replies. The side to move is computed from the board (X when the counts are equal), so there is no separate turn variable that could disagree with the board, and a client cannot make it disagree.

B. A rejection cannot change state because every check precedes the first write

Listing 4 is `game_move`. Every error `return` appears before the line `g->board[index] = side`, and the checks only read `*g`. The proof is by inspection: a path that returns an error has executed no write, so there is nothing to undo. The other commands share the shape of one condition followed by at most one state change. The order of the checks (state, range, turn, occupancy) is the order in Table III, and it is the same order the protocol promises, so the choice of NACK is deterministic.

C. Win and draw are decided by the server after each commit

After a commit, `game_evaluate` tests the eight lines (three rows, three columns, two diagonals) before testing for a full board, so a move that fills the last cell and completes a line is a win, not a draw. The server then ends the game and replies `WIN` or `DRAW`. Table V lists the checks that back the argument. They run on the PC (game core alone), except the last row, which is the board suite.

TABLE V
Evidence for the authority argument (PC unless stated).

Check	Result
Every error compared byte-for-byte (`memcmp` of the 10 bytes)	0 failures in 50,546,417 checks
All positions reachable by legal play, each tried with every index and claim	549,946 positions; 6,859,116 illegal attempts, all left the game identical
Random calls (fixed seed): six commands, indices $-3..12$	2,000,000 calls, invariants held; 22,948 wins, 3,003 draws
Independent Python brute force vs. C rules	255,168 games: 131,184 X, 77,904 O, 46,080 draws; C reproduces them
All 8 lines, both players, over the serial link	`test_each_line_wins`, 16 cases; passes on the board

TABLE VI
Bad input, the server's handling, and the test that checks it.

Bad input	Handling and response	Test
Partial frame	500 ms inactivity timer; `NACK:TIMEOUT`; remainder is a new frame	`test_partial_frame_times_out`, `test_inactivity_timer_restarts_on_each_byte`
Oversize (33+ bytes)	`NACK:TOOLONG` at byte 33, skip to LF	`test_32_byte_frame_accepted_33_rejected`, `test_oversize_frames_get_exactly_one_nack`
Garbage bytes	`NACK:BADCMD` per LF; next valid frame answered	`test_random_garbage_never_breaks_the_server`, `test_garbage_does_not_change_game`
Unknown or wrong-case command	`NACK:BADCMD`	`test_bad_command_variants`
Corrupted bytes	XOR check; `NACK:BADSUM`; never executed	`test_bad_checksum_never_executes_command`
Pipelined burst of 32+ frames	Replies lost (see text); not handled	`measure\_board.py` M4
UART errors (overrun, framing)	ISR clears flags, counts; no reply possible	not tested on the board (counters cannot be read without writing to the game UART)

The exhaustive run covers the game core only, not framing, and it ran on a PC. It uses one fixed random seed, so it is repeatable rather than broad. The link-level claim rests on the same invariants being tested through the serial port on the board (Sec. VII). Hostile clients are bounded the same way: a client that lies about the turn gets `NACK:NOTURN`, one that resends a move gets `NACK:OCCUPIED` or `NACK:NOGAME`, and none of these changes the 10 bytes.

VI. Protocol Robustness and Integrity

Every malformed input is answered by a specific NACK and followed by recovery: the framer returns to its idle state at the next LF, and the game state is never touched. Table VI maps each kind of bad input to its mechanism and test. All tests named ran on the board (100 of 100 passed, Sec. VII), except where noted.

A. A 500 ms inactivity timer abandons stalled frames

A full 32-byte frame takes about 2.8 ms at 115200 baud, so 500 ms fires only when a sender has really stalled, while a person typing in a terminal (whose bytes arrive in bursts of a line) is not cut off. The timer restarts on every byte and runs only while a frame is in progress, so an idle line costs nothing. The timeout is 500 ms in the code and the protocol. On the board the observed timeout was 501.12–503.11 ms over 20 samples (median 501.78 ms), measured from the last byte sent to the arrival of `NACK:TIMEOUT` (`board\_measurements.txt`, M2), so it is within 3.2 ms of the 500 ms specification.

B. Oversize and garbage input resynchronise at the next LF

Oversize input gets one `NACK:TOOLONG` at the 33rd byte and is then skipped silently to the next LF (`test_discarding_times_out_silently` covers the case with no LF at all). Random binary data produces one reply per LF, which the tests count, and a valid frame afterwards is answered correctly. In 100 random garbage frames, each followed by `CONNECT`, 0 recoveries failed (M5, board).

Limit found: back-to-back bursts. M4 sent N identical `STATE` frames (9 bytes each) in a single write, 30 times per N. For $N \leq 16$ (144 bytes) all replies were correct (0 of 480 missing or wrong at $N = 16$). At $N = 32$ (288 bytes) 123 of 960 replies were missing or wrong, and at $N = 64$ (576 bytes) 929 of 1920. So the one-reply-per-frame guarantee (R1) holds on the board only while the client does not pipeline more than 16 frames; our client sends one request and waits, which M3 shows is reliable. The cause is probably the 128-byte receive ring filling while the polled transmit of each reply (about 1.2 ms for 14 bytes) holds up the main loop, since the input arrives faster than replies can be sent. We did not confirm this, because the drop counter cannot be read without writing to the game UART. A larger ring or an interrupt-driven transmit would be the fix; we did not make it.

C. Integrity: XOR catches every single-bit error but not swapped bytes

We computed the checksum's limits on the PC (`checksum_analysis.txt`): 0 of 2,512 single-bit errors are missed, 11.40% of two-bit errors are missed, 100% of adjacent byte swaps are missed, and 0.417% of random bodies pass by chance. XOR is therefore adequate against a single corrupted byte on a short USB cable, and it is not a defence against an adversary. The GUI treats a reply with a bad checksum as not received.

D. The server cannot block forever

Every loop is bounded. The main loop drains the ring buffer, which holds at most 128 bytes, and never waits for input. The framer holds 32 bytes and handles one byte per call. A reply is at most 25 bytes, so the polled transmit takes about 2.2 ms. Each byte waits for the USART's transmit-empty flag, which the hardware sets within one byte time (87 µs); there is no software time-out on that wait, so a faulty USART would stall the loop, and we have not tested that case. The input timeout is a comparison of two timestamps, not a delay. No watchdog is used. Measured on the board: 2000 consecutive request/reply exchanges gave 0 wrong or missing replies at 277.1 requests/s (M3), and the request-to-reply latency for `STATE` over 300 samples was min 2.63, median 3.00, 95th percentile 3.62 and max 4.33 ms (M1). This latency is seen from the PC and includes the ST-Link USB path, so it is not the firmware's processing time alone.

TABLE VII
THE BRIEF'S REQUIRED TESTS AND THEIR RESULTS ON THE BOARD.

Requirement (brief, p. 4)	Test name	Result
`CONNECT` gives `OK`; `NEW` gives empty board	`test_connect_in_idle`, `test_new_game_returns_empty_board`	pass
Legal move changes the board as expected	`test_valid_move_updates_board`	pass
Occupied cell: `NACK:OCCUPIED`, board unchanged	`test_occupied`	pass
Out of range: `NACK:RANGE`	`test_range`	pass
Move after game over: `NACK:NOGAME`	`test_x_wins_then_no_more_moves`	pass
Scripted win and draw	`test_x_wins_...`, `test_o_wins`, `test_draw`, `test_each_line_wins`	pass
AI never illegal (if implemented)	—	n/a

VII. VALIDATION

A. One pytest suite runs on three targets

The harness uses pytest [8] and pySerial [9]. It opens the port given by `-port`: a COM port for the board, or `socket://127.0.0.1:PORT` for a Python mock server or for the real `server.c` and `game.c` compiled for the PC and exposed on a TCP socket. The tests, the GUI and the board therefore share one code path. Every test starts with a known state, and the suite needs no human action. Run with `python -m pytest tests -port COM9`. Results labelled *PC* below were not run on the board.

B. All 100 tests pass on the board; the suite detects 15 of 15 injected faults

Table VII maps each required case of the brief to a named test. Together with the state-table and robustness tests, the board run is **100 passed in 91.96 s** (29 required and win-line tests, 18 state-table tests, 53 robustness tests; `stage7_board_pytest.txt`). The same 100 tests pass on the mock and on the PC build of the real C logic (PC). The AI row is n/a because we did not implement an AI.

A passing suite proves little unless it can fail. We injected 15 faults into the mock (for example a NACK:NOTURN that flips the turn anyway, an occupied cell that is overwritten, index 9 accepted, a stalled frame that never times out); the suite caught all 15 (PC mock, `stage3_mutation_check.txt`). The first run caught 14: a fault in the 2-4-6 diagonal went unnoticed because we tested too few lines, so we added a test for all 8 lines for both players (16 cases).

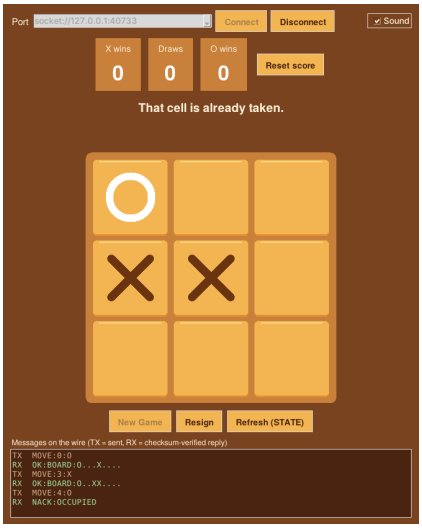


Fig. 5. GUI after a `MOVE` onto an occupied cell (mock server on a PC): `NACK:OCCUPIED`, board unchanged.

C. Screenshot: the GUI draws only what the server confirmed

Figure 5 shows the GUI after a rejected move. The status line shows the server's message and the board is as it was. The screenshot was taken against the *mock server on a PC*, not the board, since it was produced by an automated script. [Replace with a screenshot of the GUI connected to the board (COM port visible in the log).]

D. What testing revealed

- *Timestamp underflow* in the main loop (Sec. IV): found by code review before the first flash.
- *Diagonal gap* in the tests: found by the injected-fault run (above).
- *Fresh-clone build failure*: the Makefile's `mkdir build/bringup` failed when `build/` did not exist; we changed it to `mkdir -p`.
- *Read quantisation* in the test link: read latency was quantised to the 50 ms port timeout; we read `in_waiting` bytes instead.
- *Silent GUI sound*: `winsound` with `SND_MEMORY|SND_ASYNC` raised an error that our code swallowed; we now play from temporary WAV files. [Confirm you can hear it.]
- *Missing ST-Link driver* on the Windows PC made the board invisible to the flash tool; installing the driver from the IDE's drivers folder fixed it.

Not tested: unplugging the cable mid-game (the GUI's lost-reply path is exercised on the mock only), UART overrun counts on the board, and the button debounce time (30 ms is the configured value, not a measurement). [Add the LED/button check: photo or video, and how you timed the 2 Hz blink if you claim a rate.]

VIII. Conclusion

We built a Tic-Tac-Toe server on an STM32F051 that holds a 10-byte game, refuses every illegal request without changing it, and answers each frame with exactly one reply when the client waits for replies. The evidence is a 100-test suite that passes on the board in 91.96 s and also on the mock and on the C logic compiled for the PC, and a fault-injection check that catches 15 of 15 faults. The firmware image is 6,420 bytes of code in 64 KB flash, with 1,772 bytes of static RAM in 8 KB.

Limits are as follows. XOR misses all adjacent byte swaps and 11.40% of two-bit errors, so it guards against line noise, not an attacker. Pipelining is limited: when 32 frames were sent in one write, 123 of 960 replies were lost on the board (16 frames were reliable), so the guarantee of one reply per frame holds for request-and-wait clients, which is how our GUI and tests behave. The protocol supports one client and no hidden information. We did not implement the AI opponent, the move clock or undo. The overrun counters exist but cannot be read without writing to the game UART, so we do not claim a number for UART errors on the board. With more time we would add an AI opponent chosen on the server, which would need new protocol commands, and a hardware-timer move clock.

IX. Video

The demonstration video is available at [link to video]. [One or two sentences: it shows, in order, connect, legal moves, a rejected move, a win or draw, moves disabled after game over, a new game, the button and LED, and disconnect, with the board and screen in view.]

X. Code

The source code is at https://github.com/OswikaMukoma/EEE_Project_Serial_Game_Server. The repository holds the firmware (Makefile, imports into STM32CubeIDE), the Tkinter GUI, the pytest harness with the mock and PC servers, the protocol specification and the evidence files. Setup, build, flash, run and test instructions are in its `README.md`.

References

[1] Department of Electrical Engineering, University of Cape Town, "EEE3095S CS project 2026: Serial game server," Course project brief, 2026.
[2] Python Software Foundation, "`tkinter`: Python interface to Tcl/Tk," https://docs.python.org/3/library/tkinter.html.
[3] Department of Electrical Engineering, University of Cape Town, "EEE3095S course lab repository, Practical2_A HAL project base (start-up code, linker script, HAL drivers)," 2026.
[4] Anthropic, "Claude (Claude Code), used under our direction to draft code, tests, diagrams and text; every module was reviewed and run by the authors," 2026.
[5] *STM32F051x4/x6/x8 Datasheet, DocID022265 Rev 4, Table 14 (alternate functions, port A)*, STMicroelectronics.
[6] Department of Electrical Engineering, University of Cape Town, "UCT dev board schematic, v1.06," Course material.
[7] *RM0091: STM32F0x1/STM32F0x2/STM32F0x8 Advanced Arm-based 32-bit MCUs, Reference Manual*, STMicroelectronics.
[8] pytest-dev team, "pytest documentation," https://docs.pytest.org/.
[9] C. Liechti, "pySerial documentation," https://pyserial.readthedocs.io/.